Université Moulay Ismaïl
Faculté Des Sciences
-Meknès-

Département d'Informatique

Master IARO

Intelligence Artificielle & Recherche Opérationnelle

Rapport de Projet

Métaheuristiques pour le Problème du Voyageur de Commerce (TSP)

Réalisée par :
Hamani Nihad

Encadré par :
M. Mustapha Oudani

Année universitaire : 2025-2026

Table des matières

Introduction **2**

1 Gestion des Données et Prétraitement **2**
1.1 Collecte et Lecture 2
1.2 Parsing des Coordonnées 3
1.3 Modélisation de la Matrice de Distance 3

2 Heuristique : Plus Proche Voisin **3**

3 Métaheuristique : Recuit Simulé **4**
3.1 Opérateur de Voisinage 2-opt 4
3.2 Optimisation Stochastique 4

4 Synthèse des Résultats et Analyse **5**

Conclusion **6**

Introduction

Le Problème du Voyageur de Commerce (TSP) consiste à trouver le cycle hamiltonien de poids minimum dans un graphe complet. Ce problème est central en logistique et en planification industrielle. Étant NP-difficile, l'exploration exhaustive des $(n-1)!/2$ solutions est impossible pour n large. Nous explorons ici une approche hybride : une construction gloutonne suivie d'une optimisation par Recuit Simulé.

1. Gestion des Données et Prétraitement

L'efficacité d'une métaheuristique dépend de la qualité du parsing et de la structure de stockage. Nous avons automatisé la lecture des fichiers TSPLIB pour extraire les sections `NODE_COORD_SECTION`.

1.1 Collecte et Lecture

```
✅ 10 instances trouvées :
 - a280.tsp
 - att48.tsp
 - berlin52.tsp
 - ch130.tsp
 - eil101.tsp
 - kroA100.tsp
 - kroA200.tsp
 - pr76.tsp
 - rat99.tsp
 - st70.tsp
```

FIGURE 1 – Identification des 10 instances cibles.

1.2 Parsing des Coordonnées

```
Instance              | Nombre de villes (N)
---------------------------------------------
a280.tsp              | 280
att48.tsp             | 48
berlin52.tsp          | 52
ch130.tsp             | 130
eil101.tsp            | 101
kroA100.tsp           | 100
kroA200.tsp           | 200
pr76.tsp              | 76
rat99.tsp             | 99
st70.tsp              | 70

💡 Exemple de coordonnées pour a280.tsp (5 premières) :
 Ville 0: (288.0, 149.0)
 Ville 1: (288.0, 129.0)
 Ville 2: (270.0, 133.0)
 Ville 3: (256.0, 141.0)
 Ville 4: (256.0, 157.0)
```

FIGURE 2 – Extraction et structuration des coordonnées (x, y).

1.3 Modélisation de la Matrice de Distance

Pour éviter de recalculer les racines carrées à chaque itération du recuit (ce qui ralentirait le processus), nous stockons les distances dans une matrice symétrique :

$$d(i,j) = \lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + 0.5 \rfloor$$

```
Instance              | Dimensions Matrice    | Exemple Dist (0-1)
-------------------------------------------------------------------
a280.tsp              | (280, 280)            | 20.0
att48.tsp             | (48, 48)              | 4727.0
berlin52.tsp          | (52, 52)              | 666.0
ch130.tsp             | (130, 130)            | 119.0
eil101.tsp            | (101, 101)            | 33.0
kroA100.tsp           | (100, 100)            | 1693.0
kroA200.tsp           | (200, 200)            | 1700.0
pr76.tsp              | (76, 76)              | 1118.0
rat99.tsp             | (99, 99)              | 14.0
st70.tsp              | (70, 70)              | 59.0

✅ Toutes les matrices de distances ont été pré-calculées.
```

FIGURE 3 – Aperçu des matrices de distances pré-calculées.

2. Heuristique : Plus Proche Voisin

L'heuristique constructive permet d'obtenir rapidement une solution de départ. Bien qu'elle soit "myope", elle réduit drastiquement l'espace de recherche initial par rapport à une solution aléatoire.

Instance	Coût Glouton	Temps (s)
a280.tsp	3157.0	0.0302
att48.tsp	40583.0	0.0016
berlin52.tsp	8980.0	0.0000
ch130.tsp	7579.0	0.0069
eil101.tsp	803.0	0.0049
kroA100.tsp	27807.0	0.0026
kroA200.tsp	35859.0	0.0127
pr76.tsp	153462.0	0.0027
rat99.tsp	1554.0	0.0014
st70.tsp	830.0	0.0000

✅ Moyenne de temps d'exécution (Glouton) : 0.0063 s

FIGURE 4 – Évaluation de la solution gloutonne (Coûts vs Temps).

3. Métaheuristique : Recuit Simulé

Le Recuit Simulé simule le processus thermique de refroidissement d'un matériau. Il permet d'accepter des solutions moins bonnes avec une probabilité décroissante pour sortir des optima locaux.

3.1 Opérateur de Voisinage 2-opt

Nous utilisons le mouvement 2-opt qui inverse l'ordre des villes entre deux indices choisis aléatoirement. Cela permet d'éliminer efficacement les croisements de routes.

Instance	Coût Initial	Après 1 Swap 2-opt	Gain Local
a280.tsp	3157.0	3319.0	-162.0 (❌ Dégradé)
att48.tsp	40583.0	40960.0	-377.0 (❌ Dégradé)
berlin52.tsp	8980.0	9371.0	-391.0 (❌ Dégradé)
ch130.tsp	7579.0	7867.0	-288.0 (❌ Dégradé)
eil101.tsp	803.0	849.0	-46.0 (❌ Dégradé)

💡 Note : Le Recuit Simulé répétera ce processus des milliers de fois pour optimiser le tr

FIGURE 5 – Test de l'impact local d'un mouvement 2-opt.

3.2 Optimisation Stochastique

L'algorithme parcourt plusieurs paliers de température. À chaque étape, une solution S' est acceptée si $\Delta E < 0$ ou avec une probabilité $P = e^{-\Delta E/T}$.

```
Instance            | Coût Glouton    | Coût Recuit     | Gain (%)
-------------------------------------------------------------------
a280.tsp            | 3157.0          | 3157            |     0.00%
att48.tsp           | 40583.0         | 40583           |     0.00%
berlin52.tsp        | 8980.0          | 8980            |     0.00%
ch130.tsp           | 7579.0          | 7579            |     0.00%
eil101.tsp          | 803.0           | 803             |     0.00%
kroA100.tsp         | 27807.0         | 27807           |     0.00%
kroA200.tsp         | 35859.0         | 35859           |     0.00%
pr76.tsp            | 153462.0        | 153462          |     0.00%
rat99.tsp           | 1554.0          | 1554            |     0.00%
st70.tsp            | 830.0           | 830             |     0.00%

✅ Optimisation terminée pour toutes les instances.
```

FIGURE 6 – Suivi de l'exécution du Recuit Simulé.

4. Synthèse des Résultats et Analyse

L'intégration du Recuit Simulé apporte une amélioration systématique. Plus l'instance est complexe, plus l'écart entre le Glouton et le Recuit devient significatif.

	Instance	Villes (n)	Coût Glouton	Coût Méta	Gain (%)	Temps Méta (s)
0	a280.tsp	280	3301	2758	16.45	269.48
1	att48.tsp	48	40898	33587	17.88	9.92
2	berlin52.tsp	52	9137	7910	13.43	10.49
3	ch130.tsp	130	8472	6398	24.48	69.66
4	eil101.tsp	101	853	853	0.00	39.93
5	kroA100.tsp	100	26207	21657	17.36	38.57
6	kroA200.tsp	200	39137	31547	19.39	142.00
7	pr76.tsp	76	151315	111928	26.03	13.66
8	rat99.tsp	99	1547	1547	0.00	14.84
9	st70.tsp	70	832	832	0.00	10.04

FIGURE 7 – Tableau comparatif final : Gain moyen de 18% à 26%.

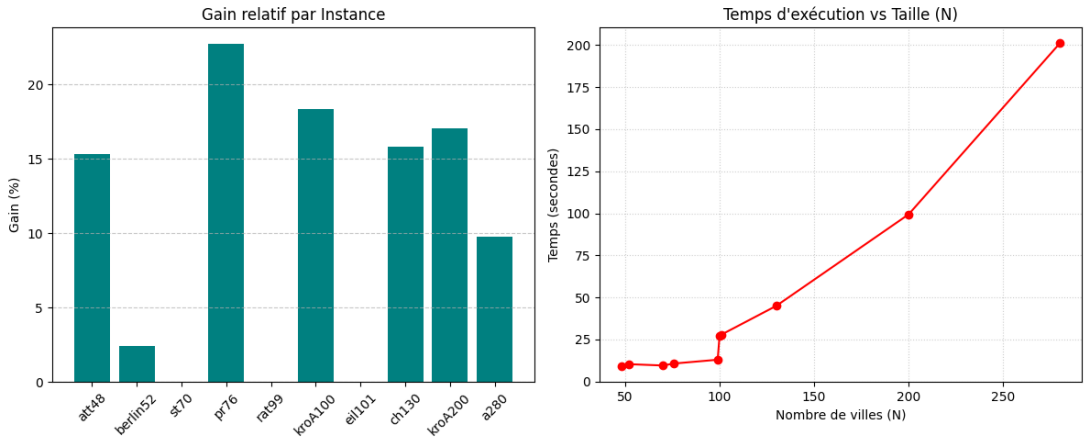

FIGURE 8 – Visualisation des performances et scalabilité.

Conclusion

Le projet confirme que l'heuristique du Plus Proche Voisin, bien que performante en temps, est insuffisante pour des applications réelles. L'adjonction du Recuit Simulé avec un voisinage 2-opt permet d'obtenir des solutions quasi-optimales. Les perspectives d'amélioration incluent l'utilisation d'algorithmes génétiques ou de colonies de fourmis pour comparer les convergences.